

多模态 Agent Post-Training 中的 优化方法分类与演化

2026 年 5 月 14 日

摘要

本项目同时涉及多模态大模型 Post-Training 和 Agent Post-Training 两个方向。本文对项目全过程中的所有优化尝试进行系统分类，阐明哪些属于多模态优化、哪些属于 Agent 优化、以及后续 RL/GRPO 方法在分类体系中的位置。核心结论：(1) 项目中唯一有效的优化全部来自多模态层面—真实 VLM 替代 mock 数据、模型规模从 3B 扩展到 7B；(2) Agent 层面的增量修补 (DPO、增强数据、微调、refusal 样本) 全部失败或持平；(3) RL/GRPO 将 Agent 优化提升到探索-最大化范式，同时通过 reward 函数将多模态质量也纳入优化目标，是突破当前瓶颈的最有希望方向。

1 概念定义

维度	多模态 Post-Training	Agent Post-Training
优化目标	VLM 对视觉输入的感知和理 解质量	模型在工具调用多步决策链 上的选择质量
训练信号	图片 → 文字 (OCR/描述/读 数) 的准确性	工具选择、格式纪律、答案恰 当性
典型数据	用户消息包含图片 + 问题	多轮对话: user→tool_call→result→...→answer
关键指标	OCR 准确度、图表读数精度、 视觉描述质量	Tool Hit, Tool Order, Format Valid, Answer Keyword
优化方法	更好的 VLM 基座、真实 VLM 标注	SFT 模仿正确轨迹、DPO 偏 好优化、RL/GRPO

表 1: 多模态 Post-Training 与 Agent Post-Training 的定义边界

关键区别：多模态优化关注模型看到了什么，Agent 优化关注模型看到了之后做什么。

2 项目中所有优化尝试的分类

2.1 多模态优化

2.1.1 核心优化：真实 VLM 替代 Mock 数据

改动：训练数据中的 vision_parse 结果从人工构造模板替换为真实 Qwen-VL 推理输出。

- Seed 数据 (46 张图片)：mock OCR 如 “OCR 结果: Invoice No=INV-1001” 替换为真实 VLM 输出如 “INVOICE\nInvoice No: INV-1001\nDate: 2026-04-01”
- 外部数据集 (751 张图片)：ChartQA/DocVQA/CORD 的占位文本 (“图表已解析; 请结合图表视觉证据...”) 替换为真实 VLM 读取的数值和文本

分类：纯多模态优化。不改 Agent 决策逻辑，不改工具调用链，只改视觉输入的标注质量。

效果：V4 keyword 从 72% 提升至 82% (eval 修正后)，这是全项目最大的单次提升。7B 上 document_validation 从 10/15 升至 15/15。

2.1.2 模型规模提升：3B→7B

改动：基座模型从 Qwen2.5-VL-3B-Instruct 换为 Qwen2.5-VL-7B-Instruct。

分类：纯多模态优化。7B 的视觉编码器更强大，OCR 和图表理解能力天然更强。Agent 逻辑 (工具调用格式) 通过相同的 SFT 数据学习，不受模型规模直接影响。

效果：document_validation 10/15→15/15, image_text_consistency 9/12→12/12。但在 Agent 侧的 tool hit 从 100% 降至 92%—更大模型的视觉理解提升伴随工具纪律下降。

2.1.3 图片分辨率限制：Resize

改动：外部数据集图片从原始大小 (最大 7.4MB) 压缩到最长 1024px。

分类：多模态优化 (虽然是负向折衷)。本质是视觉信息量与 GPU 显存的权衡。48GB 仍无法用原图训练 7B，这是多模态 Agent 训练的硬件瓶颈。

效果：训练可行但丢失了原图中的细粒度视觉信息。

2.2 Agent 优化

2.2.1 训练数据配比调整

改动：v3→v4 期间调整 recovery_repeat(2→1)、python_exec_repeat(5→2)、retrieve_docs_repeat(5→2)。

分类：Agent 优化。不改变视觉输入质量，只改变工具调用分布的偏好。

效果：v4 达到 3B 工具纪律最佳状态 (100% tool hit)，D 类冗余调用从 v3 显著降低。

2.2.2 Enforcement 机制

改动：在 runtime.py 中实现 enforce_required_tools—模型输出 final_answer 时检查是否漏调必需工具，漏了就拦截并给纠正提示。

分类：Agent 优化 (inference-time, 非训练)。强制模型遵循工具调用流程。

效果：V2 的 tool hit 从 50% 升至 100%。但其副作用在 V7B 上暴露—enforcement 阻碍了模型正确的”跳过 vision_parse 直接识别品牌”行为。

2.2.3 DPO 偏好优化（失败）

改动：三次 DPO 尝试—单轮 64 对、多轮 13 对、低 lr 微调。

分类：Agent 优化。试图优化最终答案的偏好（拒答 vs 猜测、完整答案 vs 简略）。

效果：全部导致格式崩溃（tool hit 跌至 14-60%）。DPO 在严格格式约束下失效。

失败原因：DPO 的 last-turn log-prob 优化与 Agent 的多轮决策链结构不兼容。

2.2.4 增强训练数据（V5/V6/V8/V9）

改动：

- V5: +818 条多样问法、推理链、纠错轨迹
- V6: 修 refusal 模板（从假模板换为真实 VLM 输出）
- V8: +53 条工具决策样本（47 条直接回答 +6 条 refusal+tool）
- V9: + 推理前缀版工具决策样本

分类：Agent 优化。试图通过多样化数据教模型工具选择的边界。

效果：V5 持平 (82%)、V6 退步 (70%)、V8 大幅退步 (50%)、V9 未完成。所有 Agent 增量修补均无效。

失败原因：单阶段 SFT 无法同时优化格式纪律和工具选择灵活性。

2.2.5 7B 的 LoRA Rank 搜索

改动：7B 上测试 r=16、r=32、r=64。

分类：Agent 优化（通过模型容量影响工具学习质量）。

效果：r=16 崩溃 (14%)、r=32 成功 (92%)、r=64 崩溃 (16%)。存在一个窄的最优区间。

2.3 分类汇总

优化尝试	分类	有效?	效果量级
Mock→ 真实 VLM	多模态		+10% kw
3B→7B	多模态		doc+5/15, img+3/12
原图 →Resize	多模态 (折衷)		训练可行
数据配比调整 (v3→v4)	Agent		100% tool
Enforcement	Agent		+50% tool hit(v2)
DPO	Agent	55	格式崩溃
V5 增强数据	Agent	55	持平
V6 修 refusal	Agent	55	退步
V8 工具决策	Agent	55	大幅退步
V9 推理前缀	Agent	55	未完成
7B rank 搜索	Agent(架构)	/55	r=32 有效

表 2: 所有优化尝试的分类与结果。“多模态”指仅涉及视觉输入质量，“Agent”指涉及工具调用决策。

3 核心洞察

3.1 多模态优化一致有效

每一次多模态层面的优化都带来正向收益。真实 VLM 替代 mock、3B 换 7B—这两个改动贡献了全项目 80% 以上的提升。这指向一个基础性结论：

对于 VLM Agent，视觉理解能力是 **Agent** 能力的地板。
模型能看清图片里的字，自然知道该做什么；
看不清的时候，再好的 Agent 策略也没用。

3.2 Agent 优化一致失败（增量修补范式）

在已收敛的 **SFT** 模型上做任何 **Agent** 层面的增量修补都失败。V5/V6/V8/V9/DPO×3—七个 Agent 优化尝试，无一次提升。

这不是某一个补丁的问题，是优化范式的问题：

- 单阶段 SFT 将格式纪律、工具选择、答案质量、拒答边界压缩进一个 loss
- 模型到达的“SFT 最优点”是四个目标的平衡点，不是任何单一目标的最优
- 针对任一目标的修补都会打破平衡—且修复成本远超重建成本

3.3 3B 和 7B 的能力分布差异

维度	3B (V4)	7B (V7B-clean)	差异来源
视觉理解	较弱	较强	多模态: 更大视觉编码器
工具纪律	较强	较弱	Agent: 更小模型更专注格式
数据敏感性	低	极高	Agent: 更大模型更易过拟合

表 3: 3B 与 7B 在两类能力上的互补分布

3B 和 7B 在两类能力上高度互补：3B 的工具纪律好但视觉理解弱，7B 的视觉理解强但工具纪律弱且对数据极度敏感。这一现象本身值得深入研究—为什么更大模型反而更难维持格式纪律？

4 后续优化：RL/GRPO 的分类定位

4.1 GRPO 同时优化多模态和 Agent

GRPO 的 reward 函数天然解耦多模态和 Agent 优化：

```
reward =  
vision_quality × 0.2    # 多模态: OCR/读数准确度  
+ tool_correct × 0.3    # Agent: 工具选择  
+ format_valid × 0.5    # Agent: 格式纪律  
+ answer_quality × 0.3  # Agent: 答案质量  
- tool_redundant × 0.2  # Agent: 惩罚冗余调用
```

	SFT (当前)	RL/GRPO (后续)
范式	模仿: 我给正确轨迹, 你照着学	探索: 你自己试, 我打分
多模态优化	间接: 通过标注质量提升	直接: reward 含 OCR 准确度
Agent 优化	固定: 数据怎么写模型怎么做	灵活: 模型探索不同路径
格式约束	靠数据分布	硬编码进 reward
目标冲突	四个目标抢一个 loss	加权求和, 显式 tradeoff

表 4: SFT 与 RL 在优化范式上的对比

4.2 为什么 GRPO 能解决当前瓶颈

- (1) 格式天然保护: `format_valid=-100 if parse_error` 作为硬约束写进 reward。模型探索时不可能走格式不对的路径—不像 DPO 可以降低 loss 但丢掉格式。
- (2) 工具选择不再是二元模仿: SFT 固化了”每个任务必须走某个路径”。GRPO 让模型探索多条路径, 只要 reward 高就行。比如品牌识别可以直接回答 (如果认识) 也可以调 `vision_parse` 再回答。模型通过采样发现哪些路径 reward 更高。
- (3) 多模态质量被纳入优化: SFT 无法区分”OCR 是对的但 Agent 流程错了”和”OCR 是错的但 Agent 流程对了”。GRPO 的 reward 里 `vision_quality` 项直接衡量视觉理解, 让模型同时优化感知和决策。
- (4) 数据敏感性降低: 不需要精确配比”直接回答样本”和”工具调用样本”。模型通过 reward 自己探索边界。

4.3 GRPO 的训练流程

1. 用 V4 或 V7B-clean 作为 base model (已具备基本的工具调用能力)
2. 对 50 条 eval 样本做 rollout: 让模型自由生成完整调用链
3. 根据 reward 函数对每条 rollout 打分
4. 用 GRPO loss 更新模型参数
5. 每轮采样新的 rollout, 迭代

预计迭代 5-10 轮即可看到效果。与 SFT 的 30+ 轮迭代相比, RL 的采样效率更高 (因为不需要人工构造数据, 模型自己产出训练信号)。

5 结论

1. 项目中多模态优化全部有效 (真实 VLM、7B 升级), **Agent** 增量优化全部失败 (DPO、增强数据、微调)。
2. 失败不是某个补丁的问题, 而是 SFT 作为优化范式在严格格式 + 多目标 Agent 任务上的天花板。
3. RL/GRPO 是突破天花板的路径: 它同时优化多模态和 Agent 两个维度, 将格式硬编码为

reward 约束，让模型在格式安全区内自由探索最优策略。

4. 从方法演进看，Agent Post-Training 的路径是：SFT 建立基线 → DPO 在数据充足时微调 → RL/GRPO 解锁灵活性。当前项目卡在第一步和第二步之间。